

Web 框架 VUE3 教程

讲师：燕涛

前置知识：掌握 HTML/CSS/原生 JavaScript 基础。

前端开发发展历程

前端开发经历三个典型阶段，也是我们学习的递进逻辑：

➤ 阶段 1：静态页面时代

纯 HTML+CSS 原生开发，页面无交互、无组件复用、无工程化概念，适合简单展示型页面，无法完成中大型项目开发。

➤ 阶段 2：原生 JS 交互时代

通过原生 JS 操作 DOM 实现交互，但存在大量冗余代码、DOM 操作繁琐、代码无法复用、无统一规范、维护成本极高。

➤ 阶段 3：前端工程化框架时代

以 Node.js 为底层支撑，基于 NPM 包管理、模块化开发、组件化封装、工程化构建工具，实现规范化、高效化、可维护化开发，是目前企业主流开发模式。

前端工程化开发优势

对比原生开发，工程化解决 Web 前端开发中最常见的痛点：

- 代码规范化：统一依赖管理、版本管理、项目结构。
- 模块化复用：功能拆分、组件复用，避免代码堆砌。
- 高效开发：直接使用第三方开源包，无需重复造轮子。
- 工程构建：自动打包、压缩、编译，适配生产环境。
- 生态完善：UI 库、请求库、状态管理、可视化库开箱即用。

本课程完整技术体系与学习目标

基础基建：Node.js + NPM 工程化环境。

核心框架：Vue3 组合式 API 全家桶。

路由体系：Vue Router 4 路由与权限控制。

网络交互：Axios 前后端接口请求。

UI 快速开发：Element Plus 组件库。

状态管理：Pinia 全局状态管理。

数据可视化：ECharts 图表开发。

项目落地方向：课程最终可独立开发「Vue3 后台管理系统」，包含登录鉴权、用户管理、数据 CRUD、可视化大屏、全局状态共享等完整业务功能，可直接用作课程设计、毕业设计、求职项目。

第 1 章 Node.js & NPM 工程化入门

教学目标

- 理解前端发展历程与工程化开发价值，明确 Vue3 课程整体技术体系。
- 掌握 Node.js 运行环境作用、安装与环境变量配置。
- 熟练使用 NPM 完成依赖安装、卸载、更新，理解包管理机制。
- 掌握镜像配置、常见报错排查、工程环境调试能力。
- 能够独立完成 NPM 工程化小型案例。

Node.js

让 JavaScript 无处不在

Node.js® 是一个免费、开源、跨平台的 JavaScript 运行时环境，让开发者可以创建服务器、Web 应用、命令行工具和脚本。

Node.js 内置了 npm（包管理工具），安装 Node.js 后 npm 会自动配套安装；建议选择 LTS 长期支持版（稳定），而非 Current 最新版（可能有兼容性问题）。官方下载地址：
<https://nodejs.org/zh-cn/download/>

安装步骤

➤ 步骤 1：下载安装包

打开官方下载页，选择「Windows 安装包 (.msi)」(64 位选 node-vxx.xx.x-x64.msi)；

下载完成后双击安装包，开始安装。

➤ 步骤 2：安装过程（关键配置）

点击「Next」，勾选「I accept the terms in the License Agreement」，继续「Next」；
选择安装路径（建议默认或修改为无中文 / 空格路径，如 D:\Node.js），继续「Next」；
核心步骤：勾选「Add to PATH」（自动配置环境变量，必选），其余默认，继续「Next」；
点击「Install」开始安装，等待完成后点击「Finish」。

➤ 步骤 3：验证安装

按下 Win + R，输入 cmd 打开命令提示符；

执行以下命令，输出版本号即安装成功：

```
node -v      #node 的版本
```

```
npm -v       #npm 的版本
```

注意：node 安装过程中，npm -v 不显示版本号怎样解决？

删掉.npmrc 文件就可以，这里删除不是 node.js 安装目录 npm 目录下的那个 npmrc 文件，而是在 C:\Users\用户下的.npmrc 文件，删除之后就可以了。

➤ Node.js 基础运行实操

1. 新建 test.js 文件。
2. 编写基础代码：console.log("Hello Node.js")
3. 终端执行：node test.js

NPM 包管理器

NPM (Node Package Manager) 是 Node.js 自带的**包管理工具**，是全球最大的前端开源包生态。
Vue3、Axios、Element Plus、Pinia 等所有框架和库，全部通过 NPM 安装管理。

➤ 初始化项目

```
npm init      # 一步步配置初始化  npm init -y    # 快速默认初始化（开发常用）
```

➤ 安装依赖

```
npm install 包名          # 生产依赖（项目运行需要） npm install 包名 -D
```

➤ 卸载依赖

```
npm uninstall 包名
```

➤ 更新依赖

`npm update 包名`

➤ 查看已安装依赖

`npm list`

➤ package.json 文件核心解析

项目初始化后自动生成，是**项目说明书**，核心字段：

- name: 项目名称
- version: 项目版本号
- scripts: 自定义脚本命令（启动、打包、运行）
- dependencies: 生产依赖，项目上线运行必须依赖的包
- devDependencies: 开发依赖，仅开发、编译阶段使用

➤ package-lock.json 文件作用

极易混淆两个配置文件，课堂重点区分：

- package.json: 记录依赖**大致版本**，允许小版本浮动更新。
- package-lock.json: 锁定**精确版本**、依赖依赖的版本、文件哈希值。
- 核心价值: 保证团队协作、换电脑、重新安装依赖时，所有人版本完全一致，避免项目环境不一致报错。

镜像配置与环境报错排查

默认 NPM 官方服务器在国外，国内下载速度慢、超时、报错、下载失败，必须配置国内镜像。

国内镜像推荐淘宝与华为镜像。

查看当前镜像源: `npm config get registry`

设置华为镜像:

`npm config set registry https://mirrors.huaweicloud.com/repository/npm/`

设置淘宝镜像: `npm config set registry https://registry.npmmirror.com`

恢复官方镜像: `npm config set registry https://registry.npmjs.org`

常见 NPM 报错与解决方案

报错 1: 权限不足

解决方案: 管理员身份运行终端, 或关闭文件夹只读权限

报错 2: 网络超时

解决方案: 配置淘宝镜像、重启终端、清理缓存

报错 3: 缓存冲突

```
npm cache clean --force
```

报错 4: 版本不兼容

解决方案: 更换 LTS 稳定 Node 版本, 避免最新版兼容问题

NPM 工程化案例

搭建一个简单的 Node.js 项目

步骤 1: 初始化项目

创建项目目录并进入

```
mkdir npm-demo && cd npm-demo
```

快速初始化

```
npm init -y
```

步骤 2: 安装依赖

安装生产依赖 (express)

```
npm i express
```

安装开发依赖 (nodemon, 自动重启服务)

```
npm i nodemon -D
```

步骤 3: 编写代码

创建 index.js (项目入口):

```
const express = require('express');  
  
const app = express();  
  
const port = 3000;  
  
app.get('/', (req, res) => {
```

```
    res.send('Hello npm!');  
  });  
  
  app.listen(port, () => {  
    console.log(`项目运行在 http://localhost:${port}`);  
  });
```

步骤 4：配置脚本

修改 package.json 的 scripts:

```
{  
  "scripts": {  
    "start": "node index.js",  
    "dev": "nodemon index.js"  
  }  
}
```

步骤 5：运行项目，如不进行批 4 步，则可用 node index.js 运行。

开发环境运行（修改代码自动重启）

```
npm run dev
```

访问 <http://localhost:3000>，看到 "Hello npm!" 即成功

课后作业

1. 完成 Node.js LTS 版本安装，截图展示 `node -v`、`npm -v` 版本号。
2. 配置淘宝/华为镜像。
3. 完成 npm 工程华案例。

第 2 章 Vue3 框架入门

官网: <https://cn.vuejs.org/>

渐进式 JavaScript 框架

易学易用，性能出色，适用场景丰富的 Web 前端框架。

[快速上手 →](#)[安装](#)[获取针对 Vue 2 的安全更新 ↗](#)

➤ 什么是 Vue?

Vue（发音为 /vju:/，类似 view）是一款用于构建用户界面的 JavaScript 框架。它基于标准 HTML、CSS 和 JavaScript 构建，并提供了一套声明式的、组件化的编程模型，帮助你高效地开发用户界面。无论是简单还是复杂的界面，Vue 都可以胜任。

➤ Vue 与 jQuery 的区别

Vue 以数据为核心。

jQuery 以 Dom 对象为核心。

➤ Vue 的特点

1. 遵循 **MVVM** 模式。
 2. 编码简洁，体积小，运行效率高，适合移动/PC 端开发。
 3. 它本身只关注 UI，也可以引入其它第三方库开发项目。
-
1. 动态构建用户界面的渐进式 JavaScript 框架。
 2. 作者：**尤雨溪**。

快速入门

首先通过 Vite 快速创建 Vue3 项目（Vite 是 Vue 官方推荐的新一代构建工具，启动速度远快

于 Webpack)。

1. 创建项目

```
npm create vite@latest my-vue3-project -- --template vue
```

2. 进入项目目录

```
cd my-vue3-project
```

3. 安装依赖

```
npm install
```

4. 启动项目

```
npm run dev
```

完整目录结构 Vue3+Vite

my-vue3-project/

```
├── node_modules/ # 项目依赖包（自动生成，无需手动修改）
├── public/ # 静态资源目录（不会被 Vite 编译，直接复制到打包根目录）
│   ├── favicon.ico # 网站图标
├── src/ # 核心源码目录（90%开发工作在此完成）
│   ├── assets/ # 静态资源：图片、样式、字体等（会被编译）
│   ├── components/ # 公共组件目录：存放全局复用的 Vue 组件
│   ├── App.vue # 根组件：整个应用的入口组件
│   └── main.js # 入口文件：Vue 应用的初始化入口
├── .gitignore # Git 忽略配置文件
├── index.html # 页面入口文件：Vite 的入口 HTML
├── package.json # 项目配置文件：依赖、脚本命令
├── README.md # 项目说明文档
└── vite.config.js # Vite 配置文件：代理、打包、插件等配置
```

➤ 核心文件/目录深度解析

✧ index.html（页面入口）

Vite 以`index.html`为入口，替代了传统 Webpack 的 JS 入口，直接关联 Vue 应用：

✧ main.js (Vue 应用入口文件)

核心作用：创建 Vue 实例、注册全局插件、挂载根组件，是整个 Vue 应用的启动入口。

```
JS main.js  X
src > JS main.js
1 // 1. 从vue中按需导入createApp方法
2 import { createApp } from 'vue'
3 import './style.css' //导入样式
4 // 2. 导入根组件
5 import App from './App.vue'
6 // 3. 创建应用实例并挂载到#app节点
7 createApp(App).mount('#app')
```

✧ App.vue (根组件)

Vue3 采用单文件组件 (SFC) 规范，一个.vue 文件包含模板、脚本、样式三部分：

```
▼ App.vue  X
src > ▼ App.vue > ...
1 <template>
2   <!-- 模板：HTML结构 -->
3   <div>
4     <h1>Hello Vue3</h1>
5   </div>
6 </template>
7 <script setup>
8   // 脚本：业务逻辑，setup是Vue3组合式API语法糖
9 </script>
10 <style scoped>
11   /* 样式：scoped表示样式仅作用于当前组件 */
12 </style>
```

✧ components/ 目录

存放公共可复用组件，例如`Header.vue`、`Footer.vue`，在其他组件中直接导入使用。

✧ vite.config.js (Vite 配置文件)

项目工程化核心配置文件，可配置代理、打包路径、插件等：

```
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'

// https://vite.dev/config/
export default defineConfig({
  plugins: [vue()],
  server: {
    port: 3000, // 启动端口
```

```

    open: true // 自动打开浏览器
  }
})

```

Vue3 核心优势

Vue3 是 Vue2 的全面升级版本，专为大型工程化项目设计，核心优势如下：

1. 组合式 API（Composition API）核心优势。

- 设计理念：将相同业务逻辑的代码聚合在一起，替代 Vue2 的选项式 API（data/methods/computed 分离）。
- 优势：代码可读性更高、复用性更强、大型项目维护更简单。
- 示例对比：
 - Vue2 选项式 API：数据、方法、计算属性分散编写
 - Vue3 组合式 API：同一功能代码集中在`<script setup>`中。

2. 按需导入 + 更好的 Tree-Shaking。

```

// 按需导入，而非 import Vue from 'vue'
import { createApp, ref, computed } from 'vue'

```

3. 极致性能优化

- 编译优化：模板编译生成更高效的渲染代码。
- 响应式系统重构：基于 ES6 Proxy 替代 Object.defineProperty，支持监听数组、对象新增/删除属性。
- 首屏加载更快：配合 Vite 构建工具，开发环境启动速度提升 10 倍以上。

4. 完美适配工程化开发

- 原生支持 TypeScript，类型提示更友好
- 标准化单文件组件、目录结构，适合团队协作
- 生态完善：Vite、Pinia、Vue Router 等工具链全面适配

课后作业

1. 使用 vue3+vite 构建 vue3 项目。
2. 理解 vue3 有优势，了解 vue2 与 vue3 的区别。

第 3 章 VUE3 组合式 API

Vue2 选项式 API (Options API)

Vue2 采用选项式 API, 将代码按照功能类型拆分, 统一放入固定配置项中, 核心配置包括 data、methods、computed、watch 等。

编码特点:

- 结构固定、入门简单, 适合小型项目
- 同一业务逻辑的代码被拆分到不同配置项中(数据在 data、方法在 methods、监听在 watch)
- 大型项目中代码碎片化严重, 逻辑分散, 维护、复用、重构成本极高

➤ Vue2 选项式 API 示例:

```
<script>
export default {
  data() {
    return {
      count: 0, // 业务数据
    }
  },
  methods: {
    addCount() { // 业务方法
      this.count++
    }
  }
}
</script>
```

➤ Vue3 组合式 API (Composition API)

Vue3 组合式 API 以业务逻辑为核心, 将同一个业务功能的数据、方法、监听集中编写, 实现高内聚的代码组织形式, 完美适配中大型工程化项目。

核心优势:

- 逻辑聚合: 同一业务逻辑代码集中管理, 结构清晰。
- 灵活复用: 支持自定义组合函数封装通用逻辑, 解决 Mixin 命名冲突问题。
- 类型友好: 原生适配 TS, 类型推导精准。
- 按需加载: API 按需导入, 配合 Tree-Shaking 减小打包体积。

➤ 对比总结

对比维度	选项式API (Vue2)	组合式API (Vue3)
代码组织	按代码类型拆分, 逻辑分散	按业务逻辑聚合, 高内聚
逻辑复用	依赖Mixin, 易命名冲突、溯源困难	自定义组合函数, 清晰无冲突
TS支持	支持有限, 类型推导繁琐	原生完美支持, 类型安全
适用场景	小型项目、简单页面	中大型项目、复杂业务逻辑

setup 语法糖核心详解

➤ setup 执行时机

setup 是组合式 API 的核心执行入口，其执行优先级极高：

- 执行时机：组件创建之前（beforeCreate 之前）
- 核心特点：执行时组件实例尚未创建，无法使用 this（this 为 undefined）
- 生命周期：早于所有选项式 API 生命周期钩子

➤ 标准 setup 写法 vs 语法糖写法

1) 原生 setup 写法（无语法糖）

需要手动 return 暴露变量和方法，模板才能使用，代码冗余度高。

```
<script>
export default {
  setup() {
    // 定义数据
    const count = 0
    // 定义方法
    const add = () => {
      count++
    }
    // 必须手动暴露，模板才可访问
    return { count, add }
  }
}
</script>
```

➤ setup 语法糖写法（<script setup> 推荐）

Vue3 官方推荐语法糖，无需手动 return，定义的变量、方法默认全局暴露至模板，极大简化代码。

核心优势：

- 省略繁琐的 return 语句，代码更简洁
- 自动暴露顶层变量、函数、导入的组件/方法
- 编译优化更彻底，运行性能更高
- 原生支持顶层 await，适配异步开发

```
<script setup>
// 无需 return, 模板直接使用
const count = 0
const add = () => {
  count++
}
</script>
```

➤ setup 语法规则与暴露规则

- 在 <script setup> 中：所有顶层变量、函数、导入资源，默认直接暴露给模板使用
- 非顶层（函数内部、代码块内部）变量，属于局部私有，模板无法访问
- 禁止使用 this，所有逻辑通过顶层变量编写
- 语法糖模式下，不能书写 export default，语法完全独立

Vue3 响应式核心原理（ref / reactive）

Vue3 响应式的核心：数据变更 → 自动触发视图更新。普通变量修改后不会更新视图，必须通过 Vue3 提供的响应式 API 包装，赋予数据监听能力。

➤ ref API（基本数据类型响应式）

1) 作用与原理

专门用于基本数据类型（String、Number、Boolean、Null、Undefined）的响应式包装，也可兼容简单引用类型。

底层原理：基于类 + get/set 劫持数据，通过.value 属性维护响应式数据。

2) 语法规则

```
<script setup>
import { ref } from 'vue'
// 定义响应式基本数据
const 变量名 = ref(初始值)
// 脚本中修改：必须通过 .value
// 模板中使用：直接写变量名，无需.value
</script>
```

3) 完整示例

```

<template>
  <div>
    <p>计数器: {{ count }}</p>
    <button @click="add">点击+1</button>
  </div>
</template>

<script setup>
import { ref } from 'vue'
// 创建响应式数字
const count = ref(0)

// 修改响应式数据
const add = () => {
  // 脚本内部必须通过 .value 修改值
  count.value++
}
</script>

```

➤ reactive API（引用数据类型响应式）

1) 作用与原理

专门用于引用数据类型（Object、Array）的响应式包装。

底层原理：基于 ES6 Proxy 代理对象，深度监听对象、数组的增、删、改、查操作，解决了 Vue2 Object.defineProperty 无法监听数组、对象新增属性的痛点。

2) 语法规范

```

<script setup>
import { reactive } from 'vue'
// 定义响应式对象/数组
const 变量名 = reactive(对象/数组)
// 脚本、模板中均直接操作属性，无需.value
</script>

```

3) 完整示例

```

<template>
  <div>
    <p>姓名: {{ user.name }}</p>
    <p>年龄: {{ user.age }}</p>
    <button @click="changeAge">年龄+1</button>
  </div>
</template>

```

```
<script setup>
import { reactive } from 'vue'
// 响应式对象
const user = reactive({
  name: '张三',
  age: 20
})

// 修改数据
const changeAge = () => {
  // 直接操作属性，无需.value
  user.age++
}
</script>
```

➤ ref vs reactive 核心对比

对比维度	ref	reactive
适配数据类型	优先基本数据类型，兼容简单对象/数组	专门适配引用类型（对象、数组）
读写方式	脚本内需加 <code>.value</code> ，模板省略	直接读写属性，无需 <code>.value</code>
底层原理	类封装 + get/set 劫持	ES6 Proxy 深度代理
使用场景	独立零散数据（数字、字符串、布尔值）	结构化数据（表单、列表、对象集合）
优缺点	使用简单，零散数据首选；批量数据繁琐	结构化管理清晰；直接赋值会丢失响应式

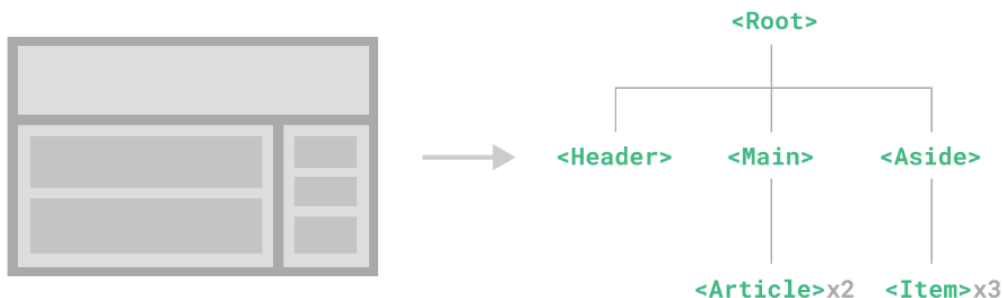
课后作业

1. 简述 setup 语法糖的执行时机和核心优势，说明为什么 setup 中不能使用 this。
2. 对比 ref 和 reactive 的底层原理、使用场景及语法差异。
3. 手写案例：实现一个个人信息编辑模块，包含姓名、性别、年龄，支持数据修改、重置。

第 4 章 Vue3 基础语法

组件基本

组件允许我们将 UI 划分为独立的、可重用的部分，并且可以对每个部分进行单独的思考。在实际应用中，组件常常被组织成层层嵌套的树状结构：



这和我们嵌套 HTML 元素的方式类似，Vue 实现了自己的组件模型，使我们可以在每个组件内封装自定义内容与逻辑。

➤ 案例：

第一步：在 components 文件夹下，创建 Base01.vue，把刚刚 App.vue 中代码拷贝过来。

第二步：还原 App.vue 中的代码。

```

<script setup>

</script>

<template>
  <h1>组件的基本使用方法</h1>
</template>

<style scoped>

</style>

```

第三步：在 App.vue 中引入组件 Base01。

```

<script setup>
  //第一步：导入组件
  import Base01 from './components/Base01.vue'
</script>

<template>
  <h1>组件的基本使用方法</h1>
  <!-- 第二步：使用组件 -->
  <Base01></Base01>
</template>

```


模板语法

➤ 文本插值

最基本的数据绑定形式是文本插值，它使用的是“Mustache”语法（即双大括号）。

```
<script setup>
  import {ref} from 'vue'
  const name = ref('李雷')
</script>

<template>
  <h1>模板语法</h1>
  <h3>{{name}}</h3>
</template>
```

➤ 原始 HTML

双大括号会将数据解释为纯文本，而不是 HTML。若想插入 HTML，你需要使用 v-html 指令。

```
<script setup>
  import {ref} from 'vue'
  const name = ref('<span style="color: red">李雷</span>')
</script>

<template>
  <h1>模板语法</h1>
  <h3>{{name}}</h3>
  <h3 v-html="name"></h3>
</template>
```

➤ Attribute 绑定

双大括号不能在 HTML attributes 中使用。想要响应式地绑定一个 attribute，应该使用 v-bind 指令。

```
<script setup>
  import {ref} from 'vue'
  import {reactive} from 'vue'
  const person = reactive({
    name: '百度',
    url: 'https://www.baidu.com'
  })
</script>

<template>
  <h1>模板语法</h1>
  <a v-bind:href="person.url">点击我去{{person.name}}搜索</a>
  <!-- 简写方式 -->
  <a :href="person.url">点击我去{{person.name}}搜索</a>
</template>
```

➤ 使用 JavaScript 表达式

Vue 实际上在所有的数据绑定中都支持完整的 JavaScript 表达式。

```
const jsname = ref('李 天 雷')

<h3>{{jsname.split(' ').reverse()}}</h3>
```

数据绑定

➤ 单向数据绑定

1. 语法: `v-bind:value="xxx"` 或简写为 `:value`。
2. 特点: 数据只能从 `data` 流向页面。

➤ 双向数据绑定

1. 语法: `v-mode:value="xxx"` 或简写为 `v-model="xxx"`。
2. 特点: 数据不仅能从 `data` 流向页面, 还能从页面流向 `data`。

```
<script setup>
  import {ref} from 'vue'
  import { reactive } from 'vue'
  const message = ref('我是VUE3')
</script>

<template>
  单向数据绑定
  <input type="text" :value="message" />
  <br>
  双向数据绑定
  <input type="text" v-model="message" />
</template>
```

事件处理

我们可以使用 `v-on` 指令 (简写为`@`) 来监听 DOM 事件, 并在事件触发时执行对应的 JavaScript。

用法: `v-on:click="methodName"` 或 `@click="handler"`。

➤ 内联事件处理器

```

<script setup>
  import { ref } from 'vue'
  const count = ref(0)
  console.log(count)
</script>

<template>
  <div>
    <h1>Hello Vue</h1>
    <div>{{ count }}</div>
    <button @click="count++">累加</button>
  </div>
</template>

```

➤ 方法事件处理器

```

<script setup>
  import { ref } from 'vue'
  import { reactive } from 'vue'
  const message = ref('VUE3的学习')
  function showInfo(event) {
    console.log("=====" + event.target.innerText);
    console.log(event)
    alert("=方法运行=")
  }
  function showInfo2(number) {
    alert("=方法运行2=" + number)
  }
</script>

<template>
  <h1>{{ message }}</h1>
  <button v-on:click="showInfo">按钮</button>
  <button @click="showInfo2(888)">按钮2</button>
</template>

```

➤ 事件修饰符

- 1 .prevent : 阻止事件的默认行为 event.preventDefault()。
- 2 .stop : 停止事件冒泡 event.stopPropagation()。

```

<script setup>
  import {ref} from 'vue'
  import { reactive } from 'vue'
  const message = ref('我是VUE3')
  function showInfo(){
    |   alert("=弹出提示信息=")
  }
</script>

<template>
  <h2>{{message}}</h2>
  <!-- 阻止默认事件（常用） -->
  <a href="https://www.baidu.com" @click.prevent="showInfo">点我提示信息</a>
  <!--阻止事件冒泡-->
  <div class="demo1" @click="showInfo">
    |   <button @click.stop="showInfo">点我提示信息</button>
  </div>
  <button @click.once="showInfo">点我提示信息</button>
</template>

<style scoped>
  .demo1{
    |   width: 200px;
    |   height: 200px;
    |   background-color: aqua;
  }
</style>

```

➤ 键盘事件

在监听键盘事件时，我们经常需要检查特定的按键。Vue 允许在 `v-on` 或 `@` 监听按键事件时添加按键修饰符。

```

<br>
<input type="text" placeholder="按下回车键给提示" @keyup.enter="showInfo" value="关键字" >

```

计算属性

1. 要显示的数据不存在，要通过计算得来。
2. 在 `computed` 对象中定义计算属性。
3. 在页面中使用 `{{方法名}}` 来显示计算的结果。

➤ 计算属性插值语法实现

```

<script setup>
  import { ref, reactive, computed } from 'vue'
  const message = ref('VUE3-计算属性')
  const author = reactive({
    firstName: '李',
    lastName: '雷'
  })
</script>

<template>
  <h1>{{ message }}</h1>
  姓: <input type="text" v-model="author.firstName">
  名: <input type="text" v-model="author.lastName">
  <br>
  姓名: <span>{{ author.firstName + author.lastName }}</span>
</template>

```

➤ 计算属性方法实现

```

<script setup>
  import { ref, reactive, computed } from 'vue'
  const message = ref('VUE3-计算属性')
  const author = reactive({
    firstName: '李',
    lastName: '雷'
  })
  function fullname() {
    return author.firstName + author.lastName;
  }
</script>

<template>
  <h1>{{ message }}</h1>
  姓: <input type="text" v-model="author.firstName">
  名: <input type="text" v-model="author.lastName">
  <br>
  姓名: <span>{{ fullname() }}</span>
</template>

```

➤ 计算属性 computed(只读)

```

<script setup>
  import { ref, reactive, computed } from 'vue'
  const message = ref('VUE3-计算属性')
  const author = reactive({
    firstName: '李',
    lastName: '雷'
  })
  const fullname = computed(() => {
    return author.firstName + author.lastName;
  })
</script>

<template>
  <h1>{{ message }}</h1>
  姓: <input type="text" v-model="author.firstName">
  名: <input type="text" v-model="author.lastName">
  <br>
  姓名: <input type="text" v-model="fullname">
</template>

```

➤ 计算属性 computed(可写)

```

<script setup>
  import { ref, reactive, computed } from 'vue'
  const message = ref('VUE3-计算属性')
  const firstName = ref('李')
  const lastName = ref('雷')
  const fullname = computed({
    get() {
      return firstName.value + '-' + lastName.value;
    },
    // setter
    set(newValue) {
      console.log("newValue is " + newValue);
      const arr = newValue.split("-")
      firstName.value = arr[0];
      lastName.value = arr[1];
    }
  })
</script>

<template>
  <h1>{{ message }}</h1>
  姓: <input type="text" v-model="firstName">
  名: <input type="text" v-model="lastName">
  <br>
  姓名: <input type="text" v-model="fullname">
</template>

```

侦听器

计算属性允许我们声明性地计算衍生值。然而在有些情况下，我们需要在状态变化时执行一些“副作用”：例如更改 DOM，或是根据异步操作的结果去修改另一处的状态。

在组合式 API 中，我们可以使用 `watch` 函数在每次响应式状态发生变化时触发回调函数。

```
<script setup>
  import { ref, computed, watch } from 'vue'
  const message = ref('VUE3-侦听器')
  const isHot = ref(true)
  const info = computed(() => {
    return isHot.value ? '热' : '冷'
  })
  function changeWeather() {
    isHot.value = !isHot.value
  }

  // 单个 ref
  watch(info, (newX) => {
    //当info的发生改变时，下面代码会执行
    console.log(`info is ${newX}`)
  })
</script>

<template>
  <h1>{{ message }}</h1>
  <h2>今天的天气比较 {{info}}</h2>
  <button @click="changeWeather">更改天气</button>
</template>
```

条件渲染

`v-if`：用于条件性地渲染一块内容。这块内容只会在指令的表达式返回真值时才被渲染。

`v-else`：可以使用 `v-else` 为 `v-if` 添加一个“else 区块”。

`v-show`

另一个可以用来按条件显示一个元素的指令是 `v-show`。其用法基本一样。不同之处在于 `v-show` 会在 DOM 渲染中保留该元素；`v-show` 仅切换了该元素上名为 `display` 的 CSS 属性。

```

<script setup>
  import {ref} from 'vue'
  const message = ref('VUE3-条件渲染')
  const isHot = ref(true)
  function change(){
    isHot.value = !isHot.value
  }
</script>

<template>
  <h1>{{ message }}</h1>
  <h2 v-if="isHot">热</h2>
  <h2 v-else>冷</h2>

  <h2 v-show="isHot">天气热</h2>
  <button @click="change">更改天气</button>
</template>

```

➤ v-if vs. v-show

v-if 是“真实的”按条件渲染，因为它确保了在切换时，条件区块内的事件监听器和子组件都会被销毁与重建。

v-if 也是惰性的：如果在初次渲染时条件值为 false，则不会做任何事。条件区块只有当条件首次变为 true 时才被渲染。

相比之下，v-show 简单许多，元素无论初始条件如何，始终会被渲染，只有 CSS display 属性会被切换。

总的来说，v-if 有更高的切换开销，而 v-show 有更高的初始渲染开销。因此，如果需要频繁切换，则使用 v-show 较好；如果在运行时绑定条件很少改变，则 v-if 会更合适。

列表渲染

我们可以使用 v-for 指令基于一个数组来渲染一个列表。v-for 指令的值需要使用 item in items 形式的特殊语法，其中 items 是源数据的数组，而 item 是迭代项的别名。

➤ 通过 key 管理状态

Vue 默认按照“就地更新”的策略来更新通过 v-for 渲染的元素列表。当数据项的顺序改变时，Vue 不会随之移动 DOM 元素的顺序，而是就地更新每个元素，确保它们在原本指定的索引位置上渲染。


```

<script setup>
  import {ref} from 'vue'
  const message = ref('VUE3-列表渲染')
  const items = ref([
    { message: 'Foo' },
    { message: 'Bar' }
  ])
</script>

<template>
  <h1>{{ message }}</h1>
  <ul>
    <li v-for="item in items">
      {{ item.message }}
    </li>
  </ul>

  <ul>
    <li v-for="(item, index) in items">
      {{ index }} - {{ item.message }}
    </li>
  </ul>
</template>

```

表单输入绑定

```

<form @submit.prevent="demo">
  账号: <input type="text" v-model="userInfo.account"> <br/><br/>
  密码: <input type="password" v-model="userInfo.password"> <br/><br/>
  年龄: <input type="number" v-model="userInfo.age" > <br/><br/>
  性别:
  男<input type="radio" name="sex" v-model="userInfo.sex" value="male">
  女<input type="radio" name="sex" v-model="userInfo.sex" value="female"> <br/><br/>
  爱好:
  学习<input type="checkbox" v-model="userInfo.hobby" value="study">
  打游戏<input type="checkbox" v-model="userInfo.hobby" value="game">
  吃饭<input type="checkbox" v-model="userInfo.hobby" value="eat">
  <br/><br/>
  所属校区
  <select v-model="userInfo.city">
    <option value="">请选择地区</option>
    <option value="beijing">北京</option>
    <option value="shanghai">上海</option>
    <option value="shenzhen">深圳</option>
    <option value="changsha">长沙</option>
  </select>
  <br/><br/>
  其他信息:
  <textarea v-model="userInfo.other"></textarea> <br/><br/>
  <input type="checkbox" v-model="userInfo.gree">阅读并接受
  <a href="">《用户协议》</a>
  <button>提交</button>
</form>

```

```

<script setup>
  import {ref, reactive} from 'vue'
  const message = ref('VUE3-表单输入绑定')
  const userInfo = reactive({
    account : '',
    password : '',
    age : 0,
    sex : 'female',
    hobby : [],
    city : 'changsha',
    other: '',
    gree : ''
  })

  function demo() {
    console.log(JSON.stringify(userInfo))
  }
</script>

```

生命周期

每个 Vue 组件实例在创建时都需要经历一系列的初始化步骤，比如设置好数据侦听，编译模板，挂载实例到 DOM，以及在数据改变时更新 DOM。在此过程中，它也会运行被称为生命周期钩子的函数，让开发者有机会在特定阶段运行自己的代码。

➤ 注册周期钩子

举例来说，onMounted 钩子可以用来在组件完成初始渲染并创建 DOM 节点后运行代码：

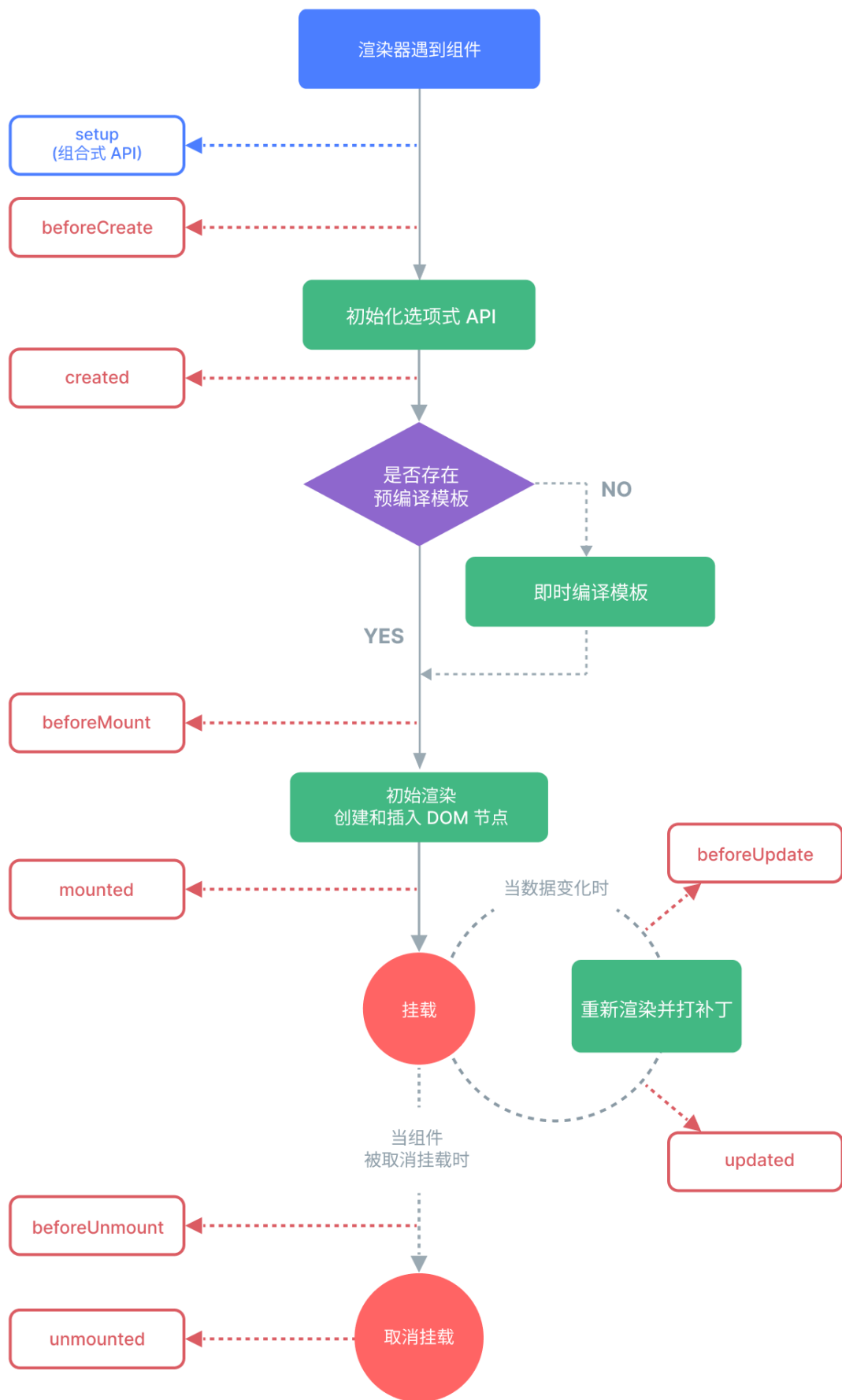
```

<script setup>
  import { onMounted } from 'vue'

  onMounted(() => {
    console.log(`the component is now mounted.`)
  })
</script>

```

当调用 onMounted 时，Vue 会自动将回调函数注册到当前正被初始化的组件实例上。这意味着这些钩子应当在组件初始化时被同步注册。



第 5 章 深入组件

回忆组件基础的内容。

➤ 组件

1. 理解：用来实现局部(特定)功能效果的代码集合(html/css/js/image...)。
2. 为什么：一个界面的功能很复杂。
3. 作用：复用编码，简化项目编码，提高运行效率。

props

作用：用于父组件给子组件传递数据。在使用 `<script setup>` 的单文件组件中，props 可以使用 `defineProps()` 宏来声明。

```
const props = defineProps(['name'])
```

name 是传递数据的属性名称。

```
<Study :name="name"></Study>
```

父组件里这样传值。

➤ 子组件：

```
<script setup>
  import {ref} from 'vue'
  const props = defineProps({
    name : String,
    age : Number,
    showName : Function
  })
</script>

<template>
  <h1>我是学生组件</h1>
  <h3>姓名: {{ name }}</h3>
  <h3>年龄: {{ age }}</h3>
  <button @click="showName">按钮</button>
</template>
```

➤ 父组件

```

<script setup>
  //第一步: 导入组件
  //import Base from './components/Base10.vue'

  import {ref} from 'vue'
  import Study from './components/Study.vue'
  const name = ref('李雷')
  const age = ref(21)

  function showName() {
    console.log("我的名字是: " + name.value+ ",今年"+age.value+"岁了");
  }
</script>

<template>
  <h1>我是父组件</h1>
  <Study :name="name" :age="age" :showName="showName"></Study>
</template>

```

➤ 组件通信

通信方式	适用场景	示例
Props	父→子	如上例
emit	子→父	子组件: <code>emit('change', 123)</code> ; 父组件: <code>@change="handleChange"</code>
provide/inject	跨级组件	父: <code>provide('key', value)</code> ; 子: <code>const val = inject('key')</code>
<u>Pinia</u>	全局状态	后续模块讲解

emit

```

<!-- 子组件 Child.vue -->
<template>
  <button @click="sendMsg">向父组件传值</button>
</template>

<script setup>
  // 定义自定义事件
  const emit = defineEmits(['msg-change'])

  const sendMsg = () => {
    // 触发事件并传参
    emit('msg-change', '来自子组件的消息')
  }
</script>

```

```
<!-- 父组件 Parent.vue -->
<template>
  <Child @msg-change="handleMsg" />
  <p>子组件消息: {{ msg }}</p>
</template>

<script setup>
import { ref } from 'vue'
import Child from './Child.vue'

const msg = ref('')
const handleMsg = (val) => {
  msg.value = val
}
</script>
```

第 6 章 路由

官网: <https://router.vuejs.org/zh/>

Vue Router Vue.js 的官方路由

为 Vue.js 提供富有表现力、可配置的、方便的路由

入门

▶ 免费视频课程

Get the Vue Router Cheat Sheet



Vue Router 是 Vue.js 的官方路由。它与 Vue.js 核心深度集成，让用 Vue.js 构建单页应用变得轻而易举。

➤ 安装

对于一个现有的使用 JavaScript 包管理器的项目，你可以从 npm registry 中安装 Vue Router:

```
$ npm install vue-router@4
```

如果你打算启动一个新项目，你可能会发现使用 create-vue 这个脚手架工具更容易，它能创建一个基于 Vite 的项目，并包含加入 Vue Router 的选项。

```
$ npm create vue@latest
```

入门

用 Vue + Vue Router 创建单页应用非常简单：通过 Vue.js，我们已经用组件组成了我们的应用。当加入 Vue Router 时，我们需要做的就是将我们的组件映射到路由上，让 Vue Router 知道在哪里渲染它们。

➤ router-link

请注意，我们没有使用常规的 a 标签，而是使用一个自定义组件 router-link 来创建链接。这使得 Vue Router 可以在不重新加载页面的情况下更改 URL，处理 URL 的生成以及编码。

➤ router-view

router-view 将显示与 URL 对应的组件。你可以把它放在任何地方，以适应你的布局。

➤ 案例：

使用 vite 新创建一个 vue 带有 router 选项的工程，体会 router-link, router-view。

```
<template>
  <header>
    

    <div class="wrapper">
      <HelloWorld msg="You did it!" />

      <nav>
        <RouterLink to="/">Home</RouterLink>
        <RouterLink to="/about">About</RouterLink>
      </nav>
    </div>
  </header>

  <RouterView />
</template>
```



You did it!

You've successfully created a project with Vite + Vue 3.

Home | About

动态路由

很多时候，我们需要将给定匹配模式的路由映射到同一个组件。例如，我们可能有一个 `User` 组件，它应该对所有用户进行渲染，但用户 ID 不同。在 `Vue Router` 中，我们可以在路径中使用一个动态字段来实现，我们称之为 路径参数：

```
const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/',
      name: 'home',
      component: HomeView
    },
    {
      path: '/about/:id',
      name: 'about',
      // route level code-splitting
      // this generates a separate chunk (About.[hash].js) for this route
      // which is lazy-loaded when the route is visited.
      component: () => import('../views/AboutView.vue')
    }
  ]
})
```

路径参数 用冒号 `:` 表示。当一个路由被匹配时，它的 `params` 的值将在每个组件中以 `this.$route.params` 的形式暴露出来。因此，我们可以通过更新 `User` 的模板来呈现当前的用户 ID：

```
<!--获得动态路由参数-->
<script setup>
import { useRouter } from 'vue-router';
const route = useRouter();
console.log(route.currentRoute.value.params);
```

除了 `$route.params` 之外，`$route` 对象还公开了其他有用的信息，如 `$route.query`（如果 URL 中存在参数）。

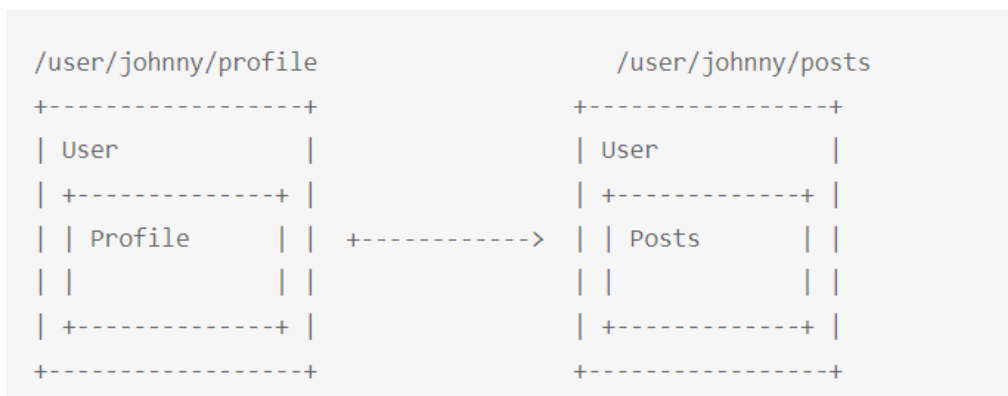
```
<RouterLink to="/about/vuejs?name=xiaoming">About</RouterLink>

console.log(route.currentRoute.value.query);
```

嵌套路由

一些应用程序的 UI 由多层嵌套的组件组成。在这种情况下，URL 的片段通常对应于特定的嵌

套组件结构，例如：



通过 Vue Router，你可以使用嵌套路由配置来表达这种关系。

```

{
  path: '/news',
  name: 'news',
  component: () => import('../views/News.vue'),
  children: [
    {
      path: '',
      name: 'news-list',
      component: () => import('../views/NewsList.vue')
    },
    {
      path: 'detail',
      name: 'news-detail',
      component: () => import('../views/NewsDetail.vue')
    }
  ]
}

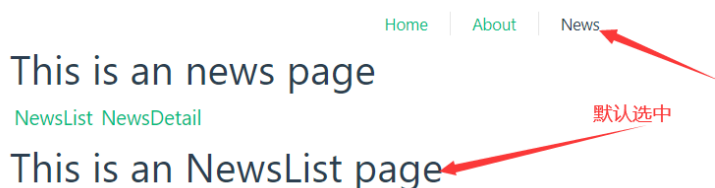
```

```

<template>
  <div class="about">
    <h1>This is an news page</h1>
    <nav>
      <router-link to="/news">NewsList</router-link>
      <router-link to="/news/detail">NewsDetail</router-link>
    </nav>
    <RouterView></RouterView>
  </div>
</template>

```

如果提供一个空的嵌套路径，则为默认选中状态。



编程式导航

除了使用 `<router-link>` 创建 `a` 标签来定义导航链接，我们还可以借助 `router` 的实例方法，通过编写代码来实现。

```
// 字符串路径
router.push('/users/eduardo')

// 带有路径的对象
router.push({ path: '/users/eduardo' })

// 命名的路由，并加上参数，让路由建立 url
router.push({ name: 'user', params: { username: 'eduardo' } })

// 带查询参数，结果是 /register?plan=private
router.push({ path: '/register', query: { plan: 'private' } })

import { useRouter } from 'vue-router';
const route = useRouter();
function goNews() {
  route.push('/about'); //导航到About页面
  //导航到AboutView页面,并带参数id为123,
  //这个是错误的: route.push({path:'/about', params:{id: '123'}});
  route.push({name:'about', params:{id: '123'}});
  //导航到AboutView页面,并带参数params.id为789,query.name为javascript
  route.push({path:'/about/789', query:{name:'javascript'}});
}
```

重定向

重定向也是通过 `routes` 配置来完成，下面例子是从 `/home` 重定向到 `/`。

```
const routes = [{ path: '/home', redirect: '/' }]
```

重定向的目标也可以是一个命名的路由：

```
const routes = [{ path: '/home', redirect: { name: 'homepage' } }]
```

第 7 章 AXIOS

Axios 是一个基于 promise 的网络请求库，可以用于浏览器和 node.js

Axios 使用简单,包尺寸小且提供了易于扩展的接口。

快速入门

官网: <https://www.axios-http.cn/>

➤ Axios 是什么?

Axios 是一个基于 promise 网络请求库，作用于 node.js 和浏览器中。它是 isomorphic 的(即同一套代码可以运行在浏览器和 node.js 中)。在服务端它使用原生 node.js http 模块，而在客户端（浏览端）则使用 XMLHttpRequests。

安装

```
$ npm install axios
```

Get 请求详见:

<https://www.axios-http.cn/docs/example>

Post 请求详见:

https://www.axios-http.cn/docs/post_example

跨域问题

1. cors 配置响应头。SpringBoot 可以用跨域注解可以解决。

麻烦后端人员，不安全，任何人都能访问。

```
@CrossOrigin
public class LoginController {
```

2. 使用代理服务器

```
server: {
  proxy: {
    '/api': {
      target: 'http://127.0.0.1:8090',
      changeOrigin: true,
      rewrite: (path) => path.replace(/^\/api/, '')
    }
  }
},
```

拦截器

在请求或响应被 then 或 catch 处理前拦截它们。

https://www.axios-http.cn/docs/config_defaults

<https://www.axios-http.cn/docs/interceptors>

第 8 章 UI 组件库

Element Plus

官网: <https://element-plus.org/zh-CN/>



Q 搜索

指南 组件 资源

Element Plus

基于 Vue 3, 面向设计师和开发者的组件库

➤ 安装

建议您使用包管理器（如 NPM、Yarn 或 pnpm）安装 Element Plus。

```
$ npm install element-plus --save
```

➤ 完整引入

如果你对打包后的文件大小不是很在乎，那么使用完整导入会更方便。在 main.js 中：

```
main.js
import './assets/main.css'

import { createApp } from 'vue'
import ElementPlus from 'element-plus'
import 'element-plus/dist/index.css'
import App from './App.vue'

const app = createApp(App)

app.use(ElementPlus)
app.mount('#app')
```

➤ 推荐自动导入

首先你需要安装 unplugin-vue-components 和 unplugin-auto-import 这两款插件。

```
$ npm install -D unplugin-vue-components unplugin-auto-import
```

➤ 然后把下列代码插入到你的 Vite 或 Webpack 的配置文件中。

```
vite.config.js
1 import { fileURLToPath, URL } from 'node:url'
2
3 import { defineConfig } from 'vite'
4 import AutoImport from 'unplugin-auto-import/vite'
5 import Components from 'unplugin-vue-components/vite'
6 import { ElementPlusResolver } from 'unplugin-vue-components/resolvers'
7 import vue from '@vitejs/plugin-vue'
8
9 // https://vitejs.dev/config/
10 export default defineConfig({
11   plugins: [
12     vue(),
13     AutoImport({
14       resolvers: [ElementPlusResolver()],
15     }),
16     Components({
17       resolvers: [ElementPlusResolver()],
18     }),
19   ],
20   resolve: {
21     alias: {
22       '@': fileURLToPath(new URL('./src', import.meta.url))
23     }
24   }
25 })
```

经过以上步骤，我们就可以在 Vue3 工程中使用 Element Plus 了。

Vant4



Vant

轻量、可定制的移动端 Vue 组件库

官网: <https://vant-ui.github.io/vant/#/zh-CN>

➤ 介绍

Vant 是一个轻量、可定制的移动端组件库，于 2017 年开源。

目前 Vant 官方提供了 Vue 2 版本、Vue 3 版本和微信小程序版本，并由社区团队维护 React 版本和支付宝小程序版本。

安装

在现有项目中使用 Vant 时，可以通过 npm 进行安装：

```
# Vue 3 项目，安装最新版 Vant
```

```
$ npm i vant
```

➤ 按需引入组件样式

<https://vant-ui.github.io/vant/#/zh-CN/quickstart>

在基于 Rsbuild、Vite、webpack 或 vue-cli 的项目中使用 Vant 时，可以使用 unplugin-vue-components 插件，它可以自动引入组件。

```
# 通过 npm 安装
```

```
npm i @vant/auto-import-resolver unplugin-vue-components unplugin-auto-import -D
```

如果是基于 Vite 的项目，在 vite.config.js 文件中配置插件：

```
import vue from '@vitejs/plugin-vue';
import AutoImport from 'unplugin-auto-import/vite';
import Components from 'unplugin-vue-components/vite';
import { VantResolver } from '@vant/auto-import-resolver';

export default {
  plugins: [
    vue(),
    AutoImport({
      resolvers: [VantResolver()],
    }),
    Components({
      resolvers: [VantResolver()],
    }),
  ],
};
```

组件及其他用法，详见官方文档。

第9章 Pinia

官方解释：Pinia 是 Vue 的存储库，它允许您跨组件/页面共享状态。

pinia 是 vuex 的升级版，因为有 vue3 新特性的加持，比 vuex 更简单、更灵活。

➤ 安装

跟随 Vue3 项目一起安装。

包管理器安装：\$ npm install pinia

package.json X

```

package.json > {} devDependencies
6   "scripts": {
7     "dev": "vite",
8     "build": "vite build",
9     "preview": "vite preview"
10  },
11  "dependencies": {
12    "element-plus": "^2.6.1",
13    "pinia": "^2.1.7",
14    "vue": "^3.4.21",
15    "vue-router": "^4.3.0"
16  },

```

Main.js 中:

JS main.js X

```

src > JS main.js > ...
1  import { createApp } from 'vue'
2  import { createPinia } from 'pinia'
3
4  import App from './App.vue'
5  import ElementPlus from 'element-plus'
6  import 'element-plus/dist/index.css'
7  import router from './router'
8
9  const app = createApp(App)
10
11 app.use(createPinia())
12 app.use(router)
13 app.use(ElementPlus)
14
15 app.mount('#app')
16

```

资源管理器 ... Login.vue JS counter.js X

打开的编辑器

src > stores > JS counter.js > ...

```

1  import { ref, computed } from 'vue'
2  import { defineStore } from 'pinia'
3
4  export const useCounterStore = defineStore('counter', () => {
5    const count = ref(0)
6    const doubleCount = computed(() => count.value * 2)
7
8    function increment() {
9      count.value++
10    }
11
12    return { count, doubleCount, increment }
13  })

```

资源管理器

- Vue-SYS
 - > .vscode
 - > node_modules
 - > public
 - > src
 - > assets
 - > components
 - > router
 - > stores
 - JS counter.js
 - > views
 - > Command.vue

页面/组件使用 pinia

```

import {useCounterStore} from '../stores/counter';
const store = useCounterStore();
const router = useRouter();

```


Store 是什么？

Store（如 Pinia）是一个保存状态和业务逻辑的实体，它并不与你的组件树绑定。换句话说，它承载着全局状态。它有点像一个永远存在的组件，每个组件都可以读取和写入它。它有三个概念，state、getter 和 action，我们可以假设这些概念相当于组件中的 data、computed 和 methods。

➤ 应该在什么时候使用 Store？

一个 Store 应该包含可以在整个应用中访问的数据。这包括在许多地方使用的数据，例如显示在导航栏中的用户信息，以及需要通过页面保存的数据，例如一个非常复杂的多步骤表单。

Pinia 案例

我们知道，pinia 可以保存全局共享数据。在实际项目中，我们可以在状态管理中保存用户数据及权限信息。

三个核心概念

➤ State

state 是 store 的核心。在 Pinia 中，state 被定义为一个返回初始状态的函数。State 类似于组件中的 data。一旦 store 被实例化，你就可以直接在 store 上访问 state、getters 和 actions 中定义的任何属性

```
import { ref, computed } from 'vue'
import { defineStore } from 'pinia'

export const useUserInfoStore = defineStore('userInfo', {
  state: () => {
    return {
      user: {
        username: "",
        password: ""
      }
    }
  },
})
```

➤ Getter

Getter 完全等同于 store 的 state 的计算值。可以通过 `defineStore()` 中的 `getters` 属性来定义它们。推荐使用箭头函数，并且它将接收 `state` 作为第一个参数。

```
getters: {
  getUsername(state) {
    return state.user.username
  },
  getPassword(state) {
    return state.user.password
  }
},
```

➤ Action

Action 相当于组件中的 `method`。它们可以通过 `defineStore()` 中的 `actions` 属性来定义，并且它们也是定义业务逻辑的完美选择。

```
actions: {
  setUsername(username) {
    this.user.username = username
  },
  setPassword(password) {
    this.user.password = password
  }
},
```

pinia 持久化存储

第一步：安装持久化插件 `pinia-plugin-persistedstate`。

```
$ npm i pinia-plugin-persistedstate
```

第二步：将插件添加到 `pinia` 实例上。

```
Info.js JS main.js x Login.vue Header.vue
JS main.js > ...
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import piniaPluginPersistedstate from 'pinia-plugin-persistedstate' //引入持久化插件

import App from './App.vue'
import ElementPlus from 'element-plus'
import 'element-plus/dist/index.css'
import router from './router'

const app = createApp(App)
const pinia = createPinia() //创建pinia实例
pinia.use(piniaPluginPersistedstate) //将插件添加到 pinia 实例上
app.use(pinia)
app.use(router)
app.use(ElementPlus)

app.mount('#app')
```

第三步：配置。

```
persist:{
  enabled:true, // 需要开启才可以持久化
  strategies:[
    {
      key:"user", // 自定义缓存的key
      paths:["user"], // 默认会缓存所有state数据，可以指定名称进行缓存
      storage:window.localStorage // 缓存地址
    }
  ]
}
```

➤ 页面使用 store

```
import {useUserInfoStore} from '../stores/userInfo';
const store = useUserInfoStore();

<el-sub-menu index="6">
  <template #title>您好!{{ store.user.username }}</template>
  <el-menu-item index="/home/userinfo">用户信息</el-menu-item>
  <el-menu-item index="/login">退出</el-menu-item>
</el-sub-menu>
```

第 10 章 ECharts

一个基于 JavaScript 的开源可视化图表库。ECharts 是一个使用 JavaScript 实现的开源可视化库，可以流畅的运行在 PC 和移动设备上，兼容当前绝大部分浏览器（IE8/9/10/11, Chrome, Firefox, Safari 等），底层依赖矢量图形库 ZRender，提供直观，交互丰富，可高度个性化定制的数据可视化图表。

➤ 通俗的理解：

1. 是一个 JS 插件。
2. 性能好可流畅运行 PC 与移动设备。
3. 兼容主流浏览器。
4. 提供很多常用图表【折线图、柱状图、散点图、饼图、K 线图】，且可定制（支持自定义）。

官网：<https://echarts.apache.org/zh/index.html>

Apache ECharts

一个基于 JavaScript 的开源可视化图表库

快速入门

所有示例



入门案例

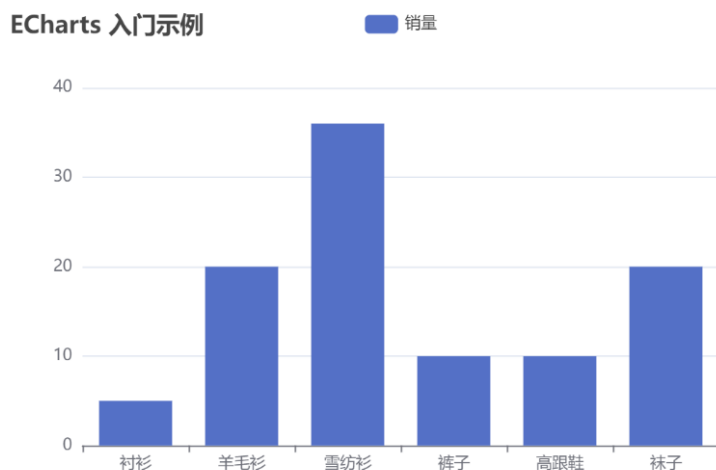
在刚才保存 echarts.js 的目录新建一个 index.html 文件，内容如下：

```
<script src="js/echarts.min.js"></script>
</head>
<body>
  <!-- 为 ECharts 准备一个定义了宽高的 DOM -->
  <div id="main" style="width: 600px;height:400px;"></div>
  <script type="text/javascript">
    // 基于准备好的dom, 初始化echarts实例
    var myChart = echarts.init(document.getElementById('main'));

    // 指定图表的配置项和数据
    var option = {
      title: {
        text: 'ECharts 入门示例'
      },
      tooltip: {},
      legend: {
        data: ['销量']
      },
      xAxis: {
        data: ['衬衫', '羊毛衫', '雪纺衫', '裤子', '高跟鞋', '袜子']
      },
      yAxis: {},
      series: [
        {
          name: '销量',
          type: 'bar',
          data: [5, 20, 36, 10, 10, 20]
        }
      ]
    };

    // 使用刚指定的配置项和数据显示图表。
    myChart.setOption(option);
  </script>
</body>
```

运行后：



Vue3 集成 ECharts

第一步：通过 npm 安装 ECharts。

```
$ npm install echarts --save
```

第二步：在 main.js 中引入 echarts。

```
main.js > ...
import { createApp } from 'vue'
import ElementPlus from 'element-plus'
import 'element-plus/dist/index.css'
import App from './App.vue'

//引入echarts
import * as echarts from 'echarts'
const app = createApp(App)

//全局挂载echarts
app.config.globalProperties.$echarts = echarts

app.use(ElementPlus)
app.mount('#app')
```

第三步：页面使用 echarts 完成柱状图。

```
<template>
  <div class="global-properties">
    <h1>柱状图</h1>
    <div ref="LineChartRef" class="echart"></div>
  </div>
</template>

<script setup>
import { ref, onMounted, getCurrentInstance } from 'vue'
const LineChartRef = ref()
onMounted(() => {
  // 获取全局echarts实例
  const echarts = getCurrentInstance().appContext.config.globalProperties.$echarts
  // 基于准备好的dom, 初始化echarts实例
  const lineChar = echarts.init(LineChartRef.value);
```

```
// 绘制图表
lineChar.setOption({
  title: {
    text: 'ECharts 入门示例'
  },
  tooltip: {},
  legend: {
    data: ['销量']
  },
  xAxis: {
    data: ['衬衫', '羊毛衫', '雪纺衫', '裤子', '高跟鞋', '袜子']
  },
  yAxis: {},
  series: [
    {
      name: '销量',
      type: 'bar',
      data: [5, 20, 36, 10, 10, 20]
    }
  ]
});
})
</script>
```

ECharts 配置语法

第一步：在 Vue3 工程中安装并引入 ECharts。

第二步：为 ECharts 准备一个具备高宽的 DOM 容器。

第三步：设置配置信息

ECharts 库使用 json 格式来配置。这里 option 表示使用 json 数据格式的配置来绘制图表。步骤如下：

➤ 标题

为图表配置标题：

```
title: {
  text: '第一个 ECharts 实例'
}
```

➤ 提示信息

配置提示信息：

```
tooltip: {},
```

➤ 图例组件

图例组件展现了不同系列的标记(symbol)，颜色和名字。可以通过点击图例控制哪些系列不显示。

```

legend: {
  data: [{
    name: '系列1',
    // 强制设置图形为圆。
    icon: 'circle',
    // 设置文本为红色
    textStyle: {
      color: 'red'
    }
  }]
}

```

➤ X 轴

配置要在 X 轴显示的项:

```

xAxis: {
  data: ["衬衫", "羊毛衫", "雪纺衫", "裤子", "高跟鞋", "袜子"]
}

```

➤ Y 轴

配置要在 Y 轴显示的项。

```
yAxis: {}
```

➤ 系列列表

每个系列通过 type 决定自己的图表类型:

```

series: [{
  name: '销量', // 系列名称
  type: 'bar', // 系列图表类型
  data: [5, 20, 36, 10, 10, 20] // 系列中的数据内容
}]

```

每个系列通过 type 决定自己的图表类型:

type: 'bar': 柱状/条形图

type: 'line': 折线/面积图

type: 'pie': 饼图

type: 'scatter': 散点（气泡）图

type: 'effectScatter': 带有涟漪特效动画的散点（气泡）

type: 'radar': 雷达图

type: 'tree': 树型图

type: 'treemap': 树型图

type: 'sunburst': 旭日图
type: 'boxplot': 箱形图
type: 'candlestick': K 线图
type: 'heatmap': 热力图
type: 'map': 地图
type: 'parallel': 平行坐标系的系列
type: 'lines': 线图
type: 'graph': 关系图
type: 'sankey': 桑基图
type: 'funnel': 漏斗图
type: 'gauge': 仪表盘
type: 'pictorialBar': 象形柱图
type: 'themeRiver': 主题河流
type: 'custom': 自定义系列

➤ ECharts 各个配置项详细说明总结:

<https://www.runoob.com/echarts/echarts-setup.html>

官方文档

- Vue3 官方文档: <https://cn.vuejs.org/guide/introduction.html>
- Vue Router 4: <https://router.vuejs.org/zh/>
- Pinia: <https://pinia.vuejs.org/zh/>
- Vite: <https://cn.vitejs.dev/>
- Echarts: <https://echarts.apache.org/zh/index.html>